

Same audio

Same audio
Any any
odd or

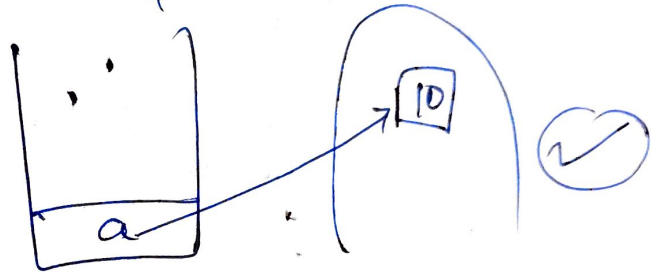
Python

① how python stores $\rightarrow$ Python treats everything as an object living in memory.

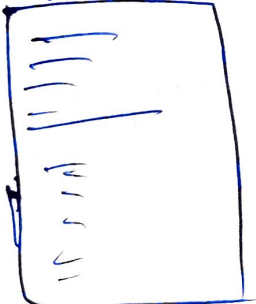
variable not any more boxes holding values

variables are the reference point to the new object.

$\boxed{36}$
 $\uparrow$ $\odot$



rajib.py

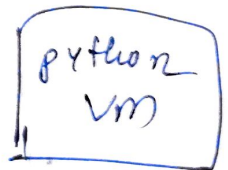


Source code

low level & platform independent

Byte code
not machine code

[mostly hidden]



PVM is called runtime engine too.

$\rightarrow$ Byte code runs faster

.pyc $\rightarrow$ compiled python

when we import, we get .pyc file,

pycache --
 $\downarrow$

folder

rajib.cpython-312.pyc

$\hookrightarrow$ version name

when we import some libes, then if we add something xtra, it doesn't auto-reload on IDLE shell.

we can use,

>>> from importlib import reload.

>>> reload(hello-chai)

Im-mutable & mutable

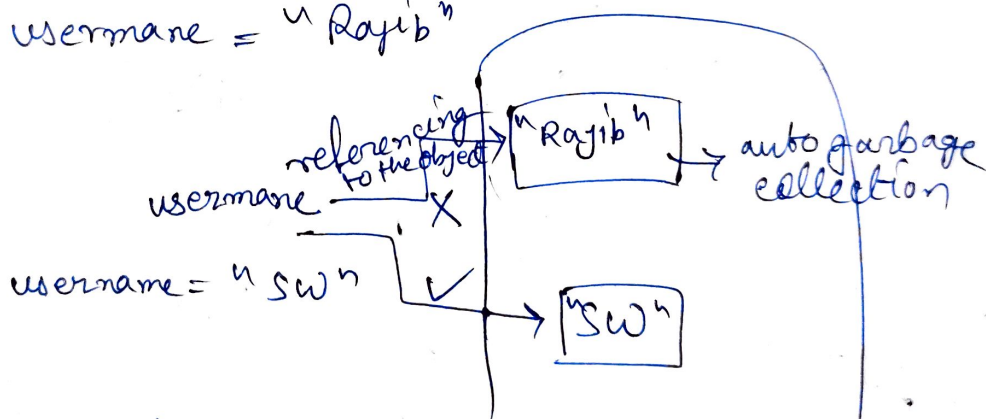
Let's create a confusion

```
x = 10
y = x
>>> x
>>> 10
y
10
```

changing $\rightarrow$ $x = 15$
o/p $x = 15$
but $y = 10$
(why?)

Suppose,

username = "Rayib"



So, when we change we never changed the "Rayib", we just referenced to "SW".

It is Im-mutable.

The reference to the memory will never change.

<u>mutable</u>	<u>Im-mutable</u>
List	Integer
Set	Floating
Dictionary	Boolean
Array	Strings
ByteArray	Tuples
	Frozen set
	Bytes

Internal working

python compiler works differently, during the time of garbage collection, when the datatype is Number or String.

for NO & string - garbage collection happens but not immediately.

$l1 = [1, 2, 3]$
 $l2 = l1$

Basically we are copying the reference of $l1$ into $l2$ both pointing to same.

$h1 = [1, 2, 3]$
 $h2 = h1[1:]$ $\rightarrow$ basically it creates the copy of the slicing value.

$==$ $\rightarrow$ compares value
 is $\rightarrow$ checks references point

Numbers in Python (Rajit Poddar) Rajit Poddar

Strings

$chai = ' ' // ' ' // ' ' // ' '$

$chai = "Lemon chai"$

$print(chai[0]) // L$

Slicing

$slice_chai = chai[0:5]$ $\rightarrow$ excluded $// Lemon$

$chai.lower()$

$chai.upper()$

$chai.strip()$ $\rightarrow$ removes extra spaces

$chai.replace("Lemon", "Ginger")$

$chai.split(",")$ (creates a list)

$chai.find("chai") // 7$

$chai.count("chai")$

before or after words.

④ check if a part/character is a part of main string.
print("Stark" in name) → True

④ Place holders.

- > chai_type = "masal"
- > quantity = 2
- > order = "I ordered { } cups of { } chai"
- > print(order.format(quantity, chai_type))
- >> I ordered 2 cups of masal chai.

> chai = "He said, \"masala chai is good\""

>> /double quotes includes.

Slicing

nums[start : stop : step]

↓ ↓

beginning ending

[includes] (excluded)

{ can be -ve

Numbers in Python

`int(2.39)`

⇒ 2

`float(40)`

⇒ 40.0

① `1 < 2 < 3` → `(1 < 2 and 2 < 3)`
⇒ True

① `1 == 2 < 3`
⇒ false → `(1 == 2 and 2 < 3)`
false

⇒ import math
• `floor()` → always goes left

• `trunc()` → always moves towards zero

⇒ complex no.

⇒ `2 + 1j`

⇒ `(2 + 1j)`

① Random

import random

`random.random()`

• `random.randint(1, 100)`

• `choice(list)`

`random.shuffle(list)`

00100 ↑
0020 → 16 → oct(64)
0xFFhex → 255 → hex(255) → 0xFF
0b1000 binary → 8 → bin(8) → 0b1000

↓
Another approach

`int('64', 8)`

⇒ 52 converted to base 8

Array / List in Python . B a o w

```
tea = ['Black', 'white', 'masala', 'oolong']
```

```
tea[1] // white .
```

```
tea[-1] // oolong
```

```
tea[2] = lemon alt tea[2:3] = ["finger"]  
o/p → changed .
```

> tea

>> Black, white, lemon, oolong

tea.append() → adds value at the end .

pop() → removes the last item .

remove("white") → will be removed

insert(posⁿ, value)

copy() // makes 2 diff ref memory .

>>> squared_num = [x**2 for x in range(10)]

>>> print(n)

o/p 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

Dictionaries

When the order doesn't matter - we use it
we use key & value pairs.

```
chai_types = { "masala" : "spicy",  
               "Lemon"  : "citrus",  
               "green"  : "mild" }
```

```
print(chai_types)
```

⑧ To access values of a key,

```
print(chai_types["masala"])
```

⑧ To add a new key value pair

```
chai_types["ginger"] = "zesty"
```

⑧ To change the value of a key

```
chai_types["lemon"] = "sour"
```

⑧ Alt method, to get the value of a key.

```
chai_types.get("lemon")
```

⑧⑧ Diff. betⁿ get & [] methods.

.get() → will return None, if the key is not found.

chai_types["lemon"] → will raise a KeyError, if - - - -

⑧ For loop

```
for chai in chai_types:
```

```
    print(chai) // only print the keys of the dict.
```

```
    print(chai_types[chai]) // will print values
```

⑧ condⁿ applies on keys only. - If lemon is in dict.

```
print("lemon" in chai_types)
```

④ pop() vs popitem()

• pop() method removes the key-value pair of the specified key & returns the value of the key.

• popitem() → removes the last key-value pair added to the dict & returns it as a tuple.

⑤ Delete a key-value pair.

```
del chai_types["ginger"]
```

⑥ Delete vs POP()

del - doesn't return the deleted value but pop does.

⑦ Nested dictionaries.

```
chai_menu = {  
    "masala": {"price": 50, "spice": "high"},  
    "ginger": { - - - - },  
}
```

to access →

```
print(chai_menu["ginger"]["spice"])
```

⑧ clear() method → used to remove all key-value pairs.

```
chai_menu.clear();
```

```
print(chai_menu) // Empty dictionary.
```

⑨ Construct a dict using a list & a default value.

```
keys = ["masala", "ginger", "lemon"]
```

```
// default_value = 2
```

```
// new_dict = dict.fromkeys(keys, default_value);
```

```
// default_value = {"price": 0, "spice": "unknown"}
```

```
// new_chai = dict.fromkeys(keys, default_value)
```


Tuple

→ List vs Tuple

List is mutable - can change the values after creating it.

Tuple is im-m - can't change.

```
my_tuple (10, 2, 3, 4, 5)
```

```
my_tuple[0] = 1 // raise a type error
```

Ⓜ we use Tuple to store related data that shouldn't be changed, for ex → 2D space co-ordinates.

Duplicate values in a Tuple

```
my_tuple = (1, 2, 3, 4, 5, 1, 2)
```

```
print(my_tuple)
```

// can use `count()` method to count the no of occurrence of a value in a tuple -

```
my_tuple.count(1) // o/p → 2
```

```
count(4) // o/p → 1
```

→ Reverse a string

```
text = "python"
```

```
rev_text = ""
```

```
for char in text:
```

```
    rev_text = char + rev_text
```

```
print(rev_text)
```

→ Optimal (slicing technique)

[start : stop : step]

↓ means go backwards.
empty → cause we want the full

```
rev_text = text[::-1]
```

```
print(rev_text)
```

② For non-repeated character

```
input_str = input("Enter the string")
```

```
for char in input_str:
```

```
    if input_str.count(char) == 1:
```

```
        print(char)
```

```
        break;
```

chai.py Working with files

```
import time
print("Rajib")
user = 42
print(user)
```

terminal

```
f = open('chai.py')
f.readline()
o/p import time\n
f.readline()
o/p print("Rajib")\n
```

flow loops works internally

using some iterable objects &

--next--() method

f.__next__()

↳ (same O/P) by internal way

if handles exception with an exception but readline() returns empty if nothing is there

myList = [1, 2, 3, 4]

i = iter(myList)

& variable

i.__next__()

~~o/p~~ 1

i.__next__()

o/p → 2

same for every loopable like range(0, 5) anything like that

Function

⑤ If we want to return multiple values.

import math

```
def circle_stats(radius):  
    area = 2 math.pi * radius2 * 2  
    cir = 2 * radius;  
    return area, cir
```

a, c = circle_stats(2)
dp print(a, " - " c)

⑥ Provide a def val to the funⁿ if no param is given.

```
def greeting(name = "User"):  
    return "Hello " + name + " ! "
```

print(greeting("Rajib")) // o/p Hello Rajib!
print(greeting()) // o/p Hello User!

⑦ Lambda funⁿ → funⁿ those have no name.

⑧ create a ~ ~ to compute cube of a no.

```
cube = lambda x, y: x + y  
x: x ** 3  
↓  
parameter  
cube(3, 5)  
cube(2)
```

⑨ Funⁿ with *args. { variable no of variable.

```
def sum_all(*args):
```

```
    // return sum(args)
```

```
    sum = 0
```

for i in args:

```
    sum = sum + i
```

```
    return sum
```

⑧ Function with ****kwargs**.

— accepts any no of keyword arguments
& prints them in format key : value

def print_kwargs(**kwargs):

for key, value in kwargs.items():
print(f"{key} : {value}")

print_kwargs(name="Shaktimal", power="lazer")

o/p name : Shaktimal
 power : lazer

⑧ Generate a funⁿ with yield

Write a funⁿ that yields even no's upto a specific limit

why we need it

def even(limit):

for i in range(2, limit+1, 2):

return i
yield i

print(even(10))

// o/p will be 2 only, if we try other methods o/p will be in list format datatype to achieve it in NO datatype only — we need
→ it keeps the memory and @ end returns it

OOP in Python
class Car: ^{capital} constructor

```
def __init__(self, brand, fuel):
```

```
    self.brand = brand
```

```
    self.fuel = fuel
```

```
def full_name(self):
```

```
    return f"{self.brand} {self.fuel}"
```

```
my_car = Car("Toyota", "diesel")
```

```
print(my_car.brand) // Toyota
```

```
print(my_car.full_name()) // Toyota Diesel.
```

Inheritance

Class Car:

```
def __init__(self, brand, model):
```

```
    self.brand = brand
```

```
    self.model = model
```

```
def full_name(self):
```

```
    return f"{self.brand} {self.model}"
```

To inherit

Class Electric_Car(Car): ^{done}

```
def __init__(self, brand, model, battery):
```

```
    self.battery = battery
```

```
    super().__init__(brand, model)
```

↓
to access the parent constructor.

Encapsulation

modify the Car class to encapsulate the brand attribute, make it private & provide a getter method.

```
def getter_brand(self):  
    return self._brand
```

self.brand = brand → to make it private, do
self._brand = brand

⑧ Static method

- can be called without creating object.
- ~~no need of self~~
- Can't directly access instance variable.
- It belongs to the class, not to the object.

@staticmethod

```
def general_desc():  
    return "cars are 4 wheelers"
```

```
print(Car.general_desc())
```

⑧ Property decorator & use a property decorator in class to make the attribute read-only.

class Car:

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.__model = model // made it private
```

@property

```
    def model(self):
```

```
        return self.__model
```

```
my_car = Car("Toyota", "safari")
```

```
my_car.model = "city" // Error
```

```
print(my_car.model()) // Error as  
                        model is working  
                        as variable
```

```
~ ~ (model)
```

Answer

multiple Inheritance .

class Battery:

```
pass  
def battery_info(self):  
    return "this is battery"
```

class Engine:

```
pass  
def engine_info(self):  
    return "this is engine"
```

class ElectricCar (Battery, Engine):

```
my_car = ElectricCar()  
print(my_car.battery_info())  
    ↖  
    ↖ engine_info()
```

Decorators

↳ Timing funⁿ Execution .

— write a decorator that measures the time a funⁿ takes to execute .

```
import time
```

```
def timetimer cate (func):  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        res = func(*args, **kwargs)  
        end = time.time()  
        return res
```

@timer — it'll go through timer funⁿ all time